SKYNET

ARQUITECTURA DE PROCESADOR 4 BITS

Versión *1.0*

06/06/2024

HISTORIAL DE VERSIONES

Version #	Implemented By	Revision Date	Approved By	Approval Date	Reason
1.0	*Rene Lozano, Osiel Gastélum*	*05/06/24*	*Dra. Nataly Medina*	*05/06/24*	Revisión final del proyecto: Prueba del procesador con programas en ensamblador; asegurar un correcto funcionamiento

UP Template Version: 12/31/07

TABLA DE CONTENIDOS

1 Introducción **3**
1.1 Propósito del documento de especificación del diseño del producto 4
2 Pautas de diseño y descripción general **4**
2.1 Supuestos / Restricciones / Estándares 5
3 Diseño de arquitectura **6**
3.1 Vista lógica 6
3.2 Micro operaciones 7
3.2.1 Suma 7
3.2.2 Resta 7
3.2.3 Multiplicación 8
3.2.4 Mover (inmediato) 8
3.2.5 Mover (directo) 8
3.2.6 Saltar 9
3.3 Buses del sistema 9
3.4 Rendimiento 9
4 Diseño del sistema **11**
4.1 Set de instrucciones 11
4.2 Descripciones detalladas de cada instrucción 12
4.2.1 Suma 12
4.2.2 Resta 12
4.2.3 Multiplicación 12
4.2.4 Mover (inmediato) 13
4.2.5 Mover (directo) 13
4.2.6 Saltar 13
4.3 Formato de datos 13
4.4 Máquinas de estados 14
4.5 Complicaciones 15

1 Introducción

1.1 Propósito del documento de especificación del diseño del producto

El presente documento se predispone como el fundamento principal detrás de la arquitectura de computadoras que se ha diseñado, esto a través de los requerimientos específicos, usos y funcionalidades. Recurriendo a conceptos definidos por William Stallings [3], una arquitectura de computadoras se refiere a todos aquellos atributos que impactan directamente la ejecución lógica de un programa.

Algunos de los fundamentos básicos que se busca suplir con el desarrollo de esta arquitectura es la capacidad de manejar datos de manera básica (movimientos y saltos en memoria) y operaciones aritméticas simples (suma, resta y multiplicación), todos bajo la premisa de un diseño derivado de la arquitectura Von Newmann.

2 Pautas de diseño y descripción general

El proyecto de diseño de una arquitectura informática implica una comprensión integral de los principios fundamentales que rigen la organización y el funcionamiento de los sistemas informáticos. Este esfuerzo implica la creación de un modelo para un sistema informático que abarque componentes de hardware, arquitectura del conjunto de instrucciones, organización de la memoria, mecanismos de entrada/salida e interconexiones del sistema. El objetivo general es desarrollar un marco coherente y eficiente que permita la ejecución de diversas tareas computacionales con un rendimiento y confiabilidad óptimos.

Algunas de las pautas a seguir en cuenta para su desarrollo son las siguientes:

1. **Requerimientos.** Se deben identificar las necesidades que la arquitectura busca satisfacer. Por ejemplo, si se utiliza meramente para operaciones aritméticas, la memoria y el conjunto de instrucciones deberá atender un enfoque hacia la ALU y las operaciones que se buscan realizar.
2. **Definición de la arquitectura base.** Es importante definir el modelo de diseño a utilizar en la arquitectura. Ejemplo de ello son las arquitecturas Harvard y Von Neumann respectivamente, las cuales establecen el cómo operan los buses, manejo de memoria, etc.
3. **Conjunto de instrucciones.** Sirve como interfaz entre el software y el hardware, definiendo y delimitando las instrucciones específicas que la computadora podrá ejecutar.
4. **Formato de datos.** Se identifica como el formato en el que se colocan los datos (en este caso en un string de bits definido), permitiendo identificar y facilitar la extracción de una parte de este para designarse como el dato, opcode, dirección, etc.
5. **Jerarquía de memoria.** Este apartado delimita el uso de cada uno de los registros o memorias de tipo ROM y el cómo se manejan internamente para aumentar la eficiencia o facilidad de escritura de datos. Por ejemplo, la partición de memoria para designar una parte al guardado de datos y otra a las instrucciones o líneas de código del programa.

6. **Diseño de la unidad de control.** Consiste en definir la máquina de estados principal que controla toda la computadora; decodifica el Opcode de las instrucciones para guiar el resto de la computadora a completar una instrucción específica.

2.1 Supuestos / Restricciones / Estándares

La arquitectura Skynet tiene como límite de entrada dos datos de 4 bits cada uno, siendo el número máximo representable con ellos un 15. Con respecto a ello, las operaciones de tipo ALU reciben siempre dos datos y una dirección, siendo cada uno de ellos de 4 bits. A continuación se presenta el tipo de formatos de instrucción manejados:

Formato de instrucción - '3'

Opcode	Dirección	Dato 1	Dato 2

Tal como se mencionó, las operaciones de tipo ALU (o de operaciones aritméticas), están diseñadas para operar directamente con los dos datos a operar y su respectiva dirección de guardado en memoria.

El diseño de este tipo de instrucciones no permite el manejo de números negativos en sus operaciones, y en caso de existir uno de estos en el resultado de alguna operación, se guardará en la bandera de números negativos.

Formato de instrucción - '2'

Opcode	Dirección	Dato

Por otro lado, las operaciones de tipo no ALU (moves y jumps) requieren de tipo de formato de instrucción de dos, donde se prioriza únicamente el dato a mover y la dirección de destino. Debe de considerarse también que existe un MOV de tipo directo, sin embargo, este opera con el mismo tipo de formato de instrucción de dos.

Formato de instrucción - '1'

Opcode	Dirección

Por último, la operación de tipo no ALU jump maneja un formato de instrucción de un elemento, siendo éste únicamente la dirección de destino a donde se quiere apuntar. Adicionalmente no requiere más que el Opcode que la unidad de control necesita para decodificar la instrucción.

3 Diseño de arquitectura

La arquitectura Skynet ejecuta un programa precargado en memoria; dependiendo del tipo de instrucción cargada y el modo de direccionamiento de la instrucción en sí, la computadora opera de distintas maneras. A continuación se definen los dos casos de interacción principales delimitados por el tipo de instrucciones manejados:

a) **Operaciones ALU.** Cuando la instrucción pertenece a la categoría de ALU, es decir, a cualquier operación de tipo aritmético como suma, resta o multiplicación, la unidad de control predefine el manejo del acumulador como parte del proceso en el diagrama de estados.

b) **Operaciones No-ALU.** Si la computadora identifica una instrucción que no es de tipo ALU, automáticamente se sabe que esta pertenece a una de transferencia de datos (move y jump).

3.1 Vista lógica

La sección de vista lógica expone los diagramas a bloques de cada una de las partes más importantes de la arquitectura, incluyendo la vista general, de la unidad de control y la partición de memoria.

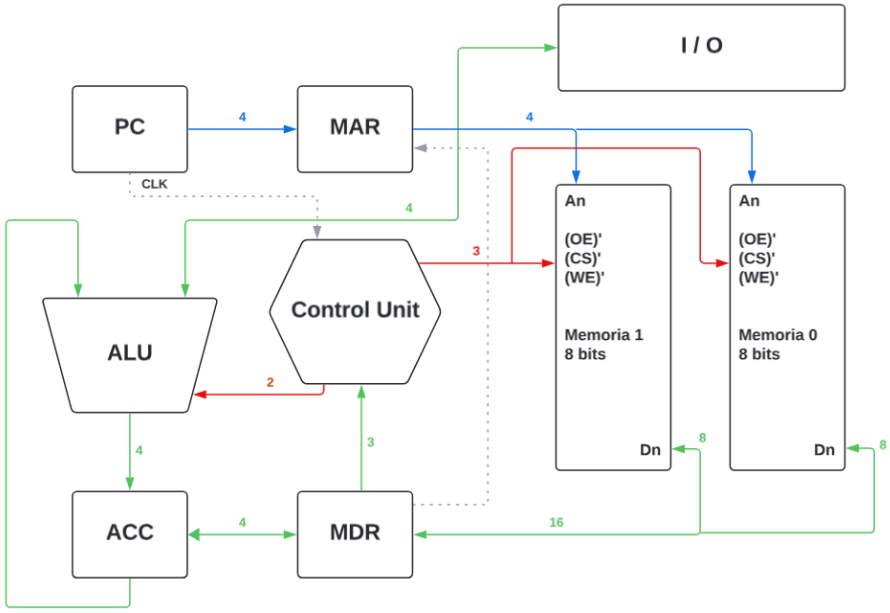

Figura 1. Diagrama de bloques de la arquitectura general de Skynet

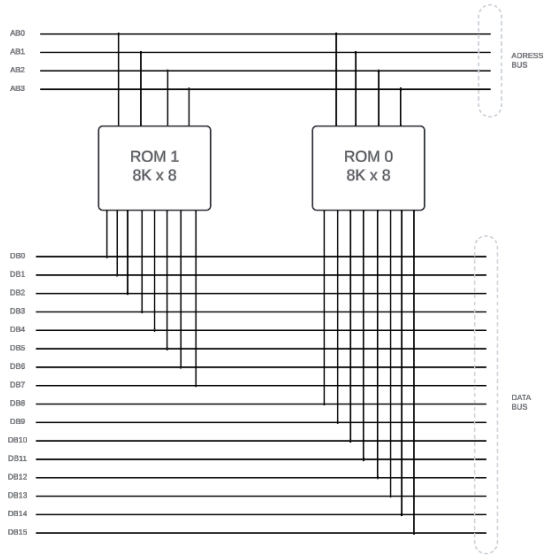

Figura 2. Conexión paralela de memorias para extensión de palabra (16 bits)

Figura 3. Diagrama simple de la partición de memoria

3.2 Micro operaciones

Este apartado describe paso a paso la metodología con la que se manejan los registros, acumulador y contador en cada una de las instrucciones con su respectivo modo de direccionamiento en caso de haberlo.

3.2.1 Suma

$$[MAR] \leftarrow [PC]$$
$$[MDR] \leftarrow [MAR], [PC] \leftarrow [PC] + 1$$
$$[ACC] \leftarrow [MDR(1)] + [MDR(2)]$$

* *MDR(n)* hace referencia al bloque del registro que almacena el dato 1 y al dato 2 respectivamente; las operaciones se realizan directamente en el acumulador y los datos se proporcionan desde el MDR tal como se mostrará a mayor detalle en el formato de datos.

3.2.2 Resta

$$[MAR] \leftarrow [PC]$$
$$[MDR] \leftarrow [MAR], [PC] \leftarrow [PC] + 1$$
$$[ACC] \leftarrow [MDR(1)] - [MDR(2)]$$

El proceso de resta permanece prácticamente idéntico al de suma, con la única diferencia en la operación aritmética como tal, la cual en este caso resta el contenido de la parte alta y baja del registro MDR.

3.2.3 MULTIPLICACIÓN

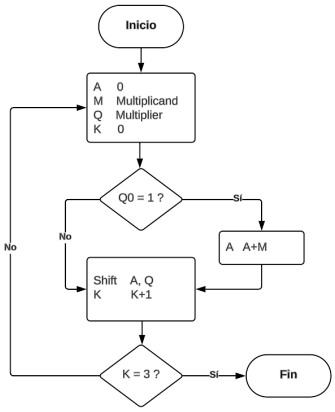

Figura 4. Diagrama de flujo del proceso de multiplicación

Debido a las características de la multiplicación y el hecho de que esta no es inmediata como la suma y resta, es más pertinente mostrar un diagrama que explique brevemente el comportamiento del mismo. En este, 'M' corresponde al dato multiplicando, 'Q' al multiplicador, 'K' al contador de *shifts* y 'A' al acumulador en la computadora.

3.2.4 MOVER (INMEDIATO)

$$[MAR] \leftarrow [PC]$$

$$[MDR] \leftarrow [MAR], [PC] \leftarrow [PC] + 1$$

$$[MAR] \leftarrow [MDR]$$

La instrucción MOV prácticamente sólo manipula los registros MDR y MAR, siendo este último el que en primera instancia le indicará a la unidad de memoria dónde se busca guardar el dato que se está enviando.

3.2.5 MOVER (DIRECTO)

$$[MAR] \leftarrow [PC]$$

$$[MDR] \leftarrow [MAR], [PC] \leftarrow [PC] + 1$$

$$[MAR] \leftarrow [MDR]$$

$$[MDR] \leftarrow [MAR]$$

En su modalidad directa, la instrucción MOV solamente varía en la etapa de búsqueda del dato; normalmente se envía directamente a MAR la dirección del dato a guardar, pero en

este caso se necesita volver a realizar una búsqueda (pasar de MDR a MAR y viceversa nuevamente) ya que lo proporcionado no es un dato sino una dirección en donde buscar el dato a transferir.

3.2.6 Saltar

$$[MAR] \leftarrow [PC]$$
$$[MDR] \leftarrow [MAR], [PC] \leftarrow [PC] + 1$$
$$[MAR] \leftarrow [MDR]$$

En cuanto a registros, la instrucción JMP los manipula prácticamente de manera idéntica a la instrucción MOV. La diferencia recae en el hecho de que la dirección que se envía a MAR es la que permite redirigir la lectura del código del programa a un punto determinado, para continuar su procesamiento a partir de ahí.

3.3 Buses del sistema

A pesar de ser una arquitectura híbrida entre Von Neumann y Harvard, la estructura de buses es muy parecida a la presente en la primera mencionada: se emplean los buses de datos, direcciones y control. El primero se encarga de transmitir únicamente datos de manera bidireccional; el segundo maneja solamente las direcciones de los datos en memoria y las direcciones de destino; el tercero lleva los bits de control correspondientes para accionar cada uno de los módulos de memoria o registros. En la siguiente figura se ilustra de mejor manera la dirección de entrada y salida de cada uno de los buses con respecto a los módulos principales de la computadora:

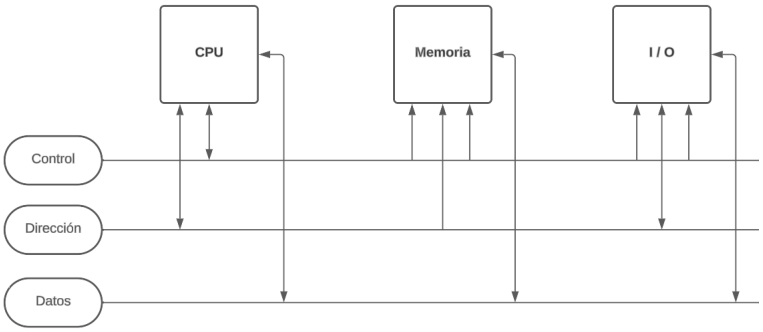

Figura 5. Diagrama de distribución de buses (control, dirección y datos)

3.4 Rendimiento

Para determinar el rendimiento del procesador, se hace uso de la fórmula que determina el número de número de ciclos por instrucción, siendo esta la siguiente:

$$CPI = \frac{\sum_{i=1}^{n} CPI_i \, x \, I_i}{Instruction\ count}$$

Con esto en mente, se obtiene la siguiente tabla con las ocurrencias de cada una de las instrucciones estandarizado a uno tomando como ejemplo un programa en donde cada instrucción tiene ocurre una vez:

Tabla 1. Ciclos de reloj requeridos para la ejecución de cada instrucción

Instruccion	Ocurrencia	Ciclos de reloj requeridos
ADD	1	1
SUB	1	1
MUL	1	4
MOV_I	1	1
MOV	1	2
JMP	1	1

Con estos datos podemos calcular el CPI:

$$CPI = \frac{(1)(1)+(1)(1)+(1)(4)+(1)(1)+(1)(2)+(1)(1)}{8}$$

$$CPI = 1.25$$

La frecuencia de operación que se maneja regularmente para el uso de la computadora ronda los 1000 Hz, por lo que se utilizará esta frecuencia para calcular el tiempo de ejecución de dicho programa de prueba.

$$CPU = \frac{(Instruction\ count\ x\ CPI)}{CLK}$$

$$CPU = \frac{(8)(1.25)}{1000\ Hz} = 10\ ms$$

Como siguiente parámetro de rendimiento, es posible calcular los millones de instrucciones por segundo que la arquitectura sería capaz de ejecutar. Esto se valora con la misma frecuencia establecida en el cálculo del CPU.

$$MIPS = \frac{(Clock\ rate)}{CPI\ x\ 10^6}$$

$$MIPS = \frac{1000\ Hz}{(1.25)\ x10^6} = 0.8x10^{-3}$$

Debido a la frecuencia operada, no es posible alcanzar el millones de instrucciones por segundo, pero esto es sencillamente solucionable si se incrementa la frecuencia a un valor mucho mayor.

Ahora, si se toma en cuenta un ciclo máquina como dos periodos de la frecuencia de reloj operación habitual (1 kHz), es posible estandarizar el tiempo de ejecución de cada instrucción:

$$f_{mc} = \frac{1000\,Hz}{2} = 500\,Hz$$

$$t_{mc} = \frac{1}{f_{mc}} = 2\,ms$$

Tabla 2. Tiempo de ejecución en milisegundos de cada instrucción

Instruccion	Ciclos máquina	Tiempo de ejecución (ms)
ADD	1/2	1
SUB	1/2	1
MUL	2	4
MOV_I	1/2	1
MOV	1	2
JMP	1/2	1

4 Diseño del sistema

4.1 Set de instrucciones

El siguiente apartado expone de manera detallada el conjunto de instrucciones en la arquitectura Skynet, al igual que el opcode que le corresponde dentro del formato de datos:

Tabla 3. Conjunto de instrucciones de la arquitectura Skynet

Instrucción	Opcode (3 bits)	Formato de datos				Descripción
ADD	000	Opcode	Destino	Dato 1	Dato 2	Suma dos números de 4 bits
SUB	001	Opcode	Destino	Dato 1	Dato 2	Resta dos números de 4 bits
MUL	010	Opcode	Destino	Dato 1	Dato 2	Multiplica dos números de 4 bits

MOV_I	011	Opcode	Destino	Dato	-	Cambia de dirección de memoria un dato
MOV	100	Opcode	Destino	Dato	-	Cambia de dirección de memoria un dato (directo).
JMP	101	Opcode	Destino	-	-	Redirige la lectura del programa a una sección de memoria

4.2 Descripciones detalladas de cada instrucción

El siguiente apartado proporciona una descripción más profunda del funcionamiento de cada instrucción, al igual que algunos ejemplos de instrucciones en lenguaje ensamblador basados en la arquitectura 8051.

4.2.1 Suma

Utilizando la abreviatura ADD, es una operación de tipo ALU que tiene como función realizar una operación aritmética de suma. Tal como se mencionó anteriormente tiene un tipo de formato de instrucción de '3' por lo que requiere de dos datos y una dirección para almacenarse.

ADD 16H, #04, #05

** Sumar de manera inmediata los datos 4 y 5, guardando en la ubicación 16H de memoria.*

4.2.2 Resta

Utilizando la abreviatura SUB, es una operación de tipo ALU que tiene como función realizar una operación aritmética de resta. Al igual que la suma, tiene un tipo de formato de instrucción de '3' por lo que requiere de dos datos y una dirección para almacenarse.

SUB 16H, #10, #2

** Restar de manera inmediata los datos 10 y 2 respectivamente, guardando en la ubicación 16H de memoria.*

Vale la pena aclarar que, para el caso de la suma, operar directamente con números negativos será imposible; en caso de resultar un número negativo después de una resta, el bit característico del complemento a '1' permitirá visualizar una bandera o *flag*.

4.2.3 Multiplicación

Como última instrucción aritmética, MUL es una operación de tipo ALU que tiene como función realizar una operación de multiplicación. De la misma manera que el resto, requiere de ser indicados ambos datos a operar y la dirección en ROM para su almacenamiento.

MUL 16H, #3, #4

** Multiplicar de manera inmediata los datos 3 y 4 respectivamente, guardando en la ubicación 16H de memoria.*

Al igual que con la resta, en caso de existir *overflow* al terminar la multiplicación, el bit de flag correspondiente será almacenado en un registro.

4.2.4 Mover (inmediato)

Como primera instrucción de tipo no ALU, MOV_I permite relocalizar en memoria un dato en específico de forma inmediata. A diferencia del resto de operaciones, este utiliza un formato de instrucción de '2', por lo que solamente necesita el dato y la dirección.

MOV_I 16H, #20

** Mover el dato 20 a la ubicación de memoria 16H*

4.2.5 Mover (directo)

A diferencia del MOV_I, el MOV tiene un modo de direccionamiento directo, por lo que en lugar de proporcionarse un dato a relocalizar, se deberá introducir la dirección en memoria de un dato a relocalizar. Su sintaxis se denota por la falta de gato (#) en el dato (siendo en este caso una dirección).

MOV 16H, 10H

** Mover el dato dentro de 10H a la dirección 16H*

4.2.6 Saltar

Como última instrucción de tipo no ALU, el JMP permite reubicar la lectura en memoria por parte del PC (Program Counter) para continuar en otro punto; tal como el nombre lo indica, permite realizar un "salto" entre partes del programa cargado.

JMP 05H

** Saltar desde la ubicación presente hasta 0H en la memoria de programa.*

Es importante aclarar que esta instrucción lleva un tipo de formato de '1', por lo que solo se requiere la dirección de memoria a la que se quiere redirigir la lectura del programa correspondiente.

4.3 Formato de datos

Debido a la naturaleza del diseño del procesador, se requiere un tamaño de palabra de 16 bits, característica que ya se había planteado anteriormente con la configuración de memoria paralela para lograr la extensión de palabra. Este tamaño permite manejar todo en un mismo lugar, facilitando a los registros el acceso a los datos que requieren. Cabe mencionar que, dado que se manejan 4 bits para ambos datos, dirección y tres bits para el opcode, existe un bit que será inutilizado.

Tabla 4. Formato de datos en arquitectura Skynet

**NU	Opcode			Dirección				Dato no. 1				Dato no. 2			
X															

** Hace referencia a la terminología en inglés Non-usable, refiriéndose al bit que se tiene como extra sin una función determinada.

4.4 Máquinas de estados

En el siguiente apartado se presentan los diagramas de estado en el software Quartus que indican el funcionamiento de la unidad de control y su relación con las unidades de memoria y la ALU.

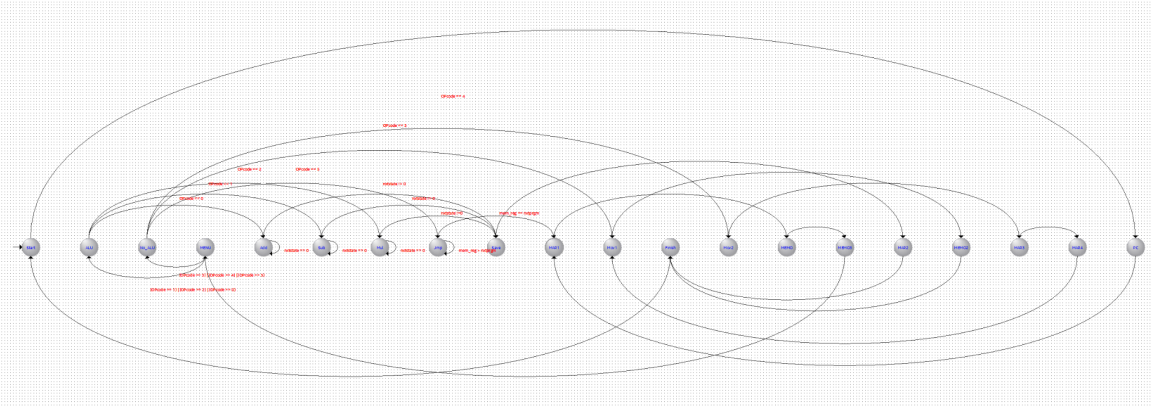

Figura 6. Máquina de estados

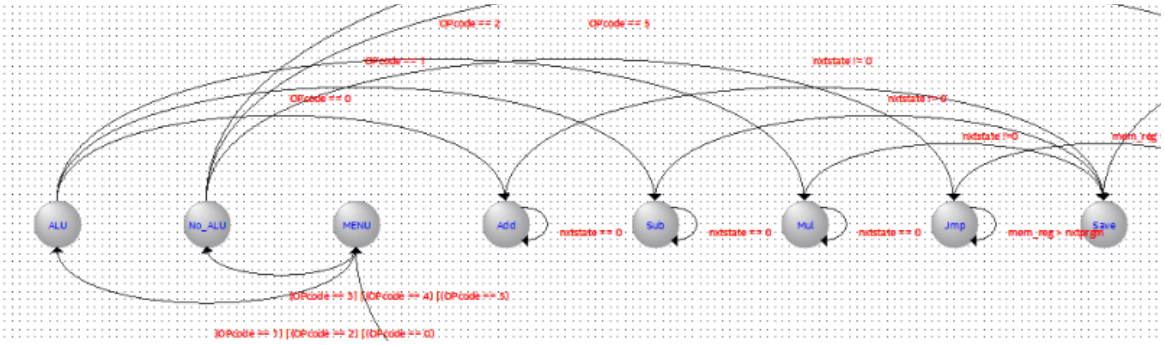

Figura 7. Vista cercana a ALU.

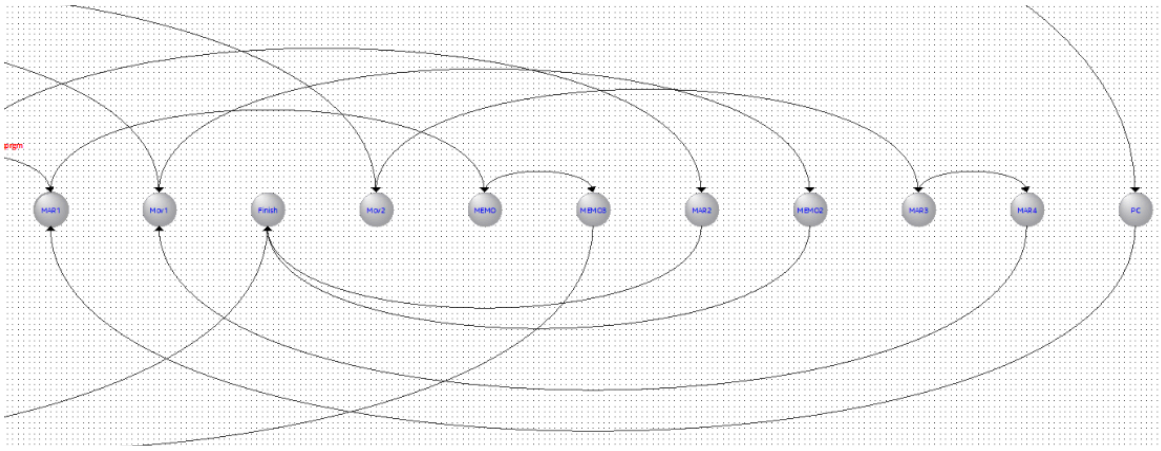

Figura 8. Vista cercana a la comunicación entre unidad de control y memoria.

4.5 Complicaciones

El desarrollo del código fue un arduo proceso que involucró la programación y revisión de las memorias para comprobar el comportamiento de la unidad de control. El código fue desarrollado a un cierto punto en el que se incluían todas las funciones, las cuales no funcionaron en el primer intento. Con ayuda de pausas en el sistema para visualizar los datos en momentos específicos, más asignaciones de valores a LEDs integrados en el FPGA, se pudo trabajar en corregir los errores y lograr un funcionamiento correcto.

Desde valores que no retenían sus datos, hasta estados que ciclaran sin fin, se encontraron bastantes complicaciones, los cuales necesitaron una dedicación de tiempo y concentración para corregir. Por ejemplo, la función MOV_I no era capaz de retener la dirección de destino; también, la multiplicación no lograba terminar, lo cual atoraba el funcionamiento del código. En fin, después de un combinado esfuerzo de 3 días, cuatro de seis funciones lograron funcionar a la perfección. Las funciones que no se pudieron completar son MUL y MOV. A pesar de cambios en el comportamiento de los cuatro corrimientos en el registro, el resultado final no siempre era correcto. Desde errores con el código Verilog, hasta problemas con la lógica, la función no se logró corregir. MOV presentó un enigmático comportamiento, el cual no fue posible descifrar, por lo cual no fue posible encontrar una solución.

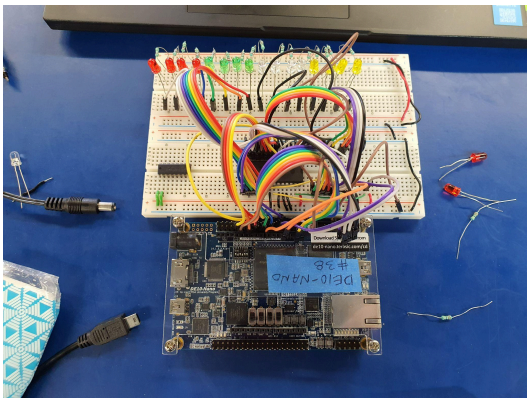

Figura 9. Fotografía de FPGA conectado a memorias y LEDs.

Apéndice A: Referencias

The following table summarizes the documents referenced in this document.

Document Name and Version	Description	Location
[1] UCM. (s.f.). Tema 10: Buses de comunicación	*El documento describe alguna de las características más importantes de la estructura fundamental detrás de un bus, al igual que los tipos que se encuentran mayormente normalizados.*	*https://www.fdi.ucm.es/profesor/jjruz/WEB2/Temas/EC10.pdf*
[2] Chávez, N. E. (s.f.). Unidad Aritmética Lógica (ALU).	*Presentación de UNAM que explica brevemente el concepto de ALU y algunos de sus ejemplos para diseño.*	*http://profesores.fi-b.unam.mx/normaelva/ALU.pdf*
[3] Stallings, W. (2006). *Organización y arquitectura de computadoras (7ma edición).*	*El libro trata sobre la estructura y funcionamiento de las computadoras, siendo su principal objetivo presentar la naturaleza y características de las computadoras actuales.*	*https://www.apps4two.com/curso_dba/bibliografia/1-Organizacion%20y%20arquitectura%20de%20computadores%2C%207a%20Ed.pdf*
[4] Ashtari, H. (2024). *Von Neumann Architecture vs. Harvard Architecture: A Detailed Comparison*	*Esta publicación explora las principales diferencias entre los dos tipos más comunes de arquitecturas de computadora.*	*https://www.spiceworks.com/tech/tech-general/articles/von-neumann-architecture-vs-harvard-architecture/*

Apéndice B: Términos clave

La siguiente tabla proporciona las definiciones de términos relevantes en este documento.

Término	Definición
Big Endian	*Formato de datos computacional que indica como estándar el acomodo de los bits más significativos a la izquierda y los menos significativos a la derecha (MSB-LSB). Es usado por algunas arquitecturas de Apple pero no se considera el estándar general.*
Bus	*Un bus se define como un medio o trayectoria de comunicación constituido por líneas (conductores) que facilita la transferencia de información, datos o entradas en un computador [1].*
ALU	*La Unidad Aritmética Lógico (ALU por sus siglas en inglés), es un circuito digital que tiene como objetivo calcular operaciones aritméticas [2].*
MAR	*También conocido como Registro de Dirección de Memoria (Memory Address Register por sus siglas en inglés), permite especificar la dirección en memoria de la palabra que será escrita [3].*
MDR	*También conocido como MBR, es un buffer que sirve de nexo entre la memoria y la máquina de estados principal; contiene los datos que deben ser procesados por la unidad de control o que tienen que ser almacenados en memoria [3].*
PC	*Definido como Contador de Programa (Program Counter en inglés), es un registro que contiene la dirección de la próxima pareja de instrucciones que serán leídas en memoria [3].*
ACC	*El acumulador es un registro que se emplea para almacenar operandos y resultados de la ALU temporalmente. Por ejemplo, al realizar una multiplicación de dos números de 4 bits cada uno, el resultado de 8 bits será plasmado en el acumulador para después transferirse a un registro o dirección de memoria correspondiente [3].*